

Bioinformática

Transformada de Burrows-Wheeler

Rodrigo Santamaría

BWT

- **Estructuras de datos**
- Transformada de Burrows-Wheeler
- Búsqueda de patrones

Estructuras de datos

Tries

Tries de sufijos

Árboles de sufijos

Arrays de sufijos

Buscando patrones en genomas

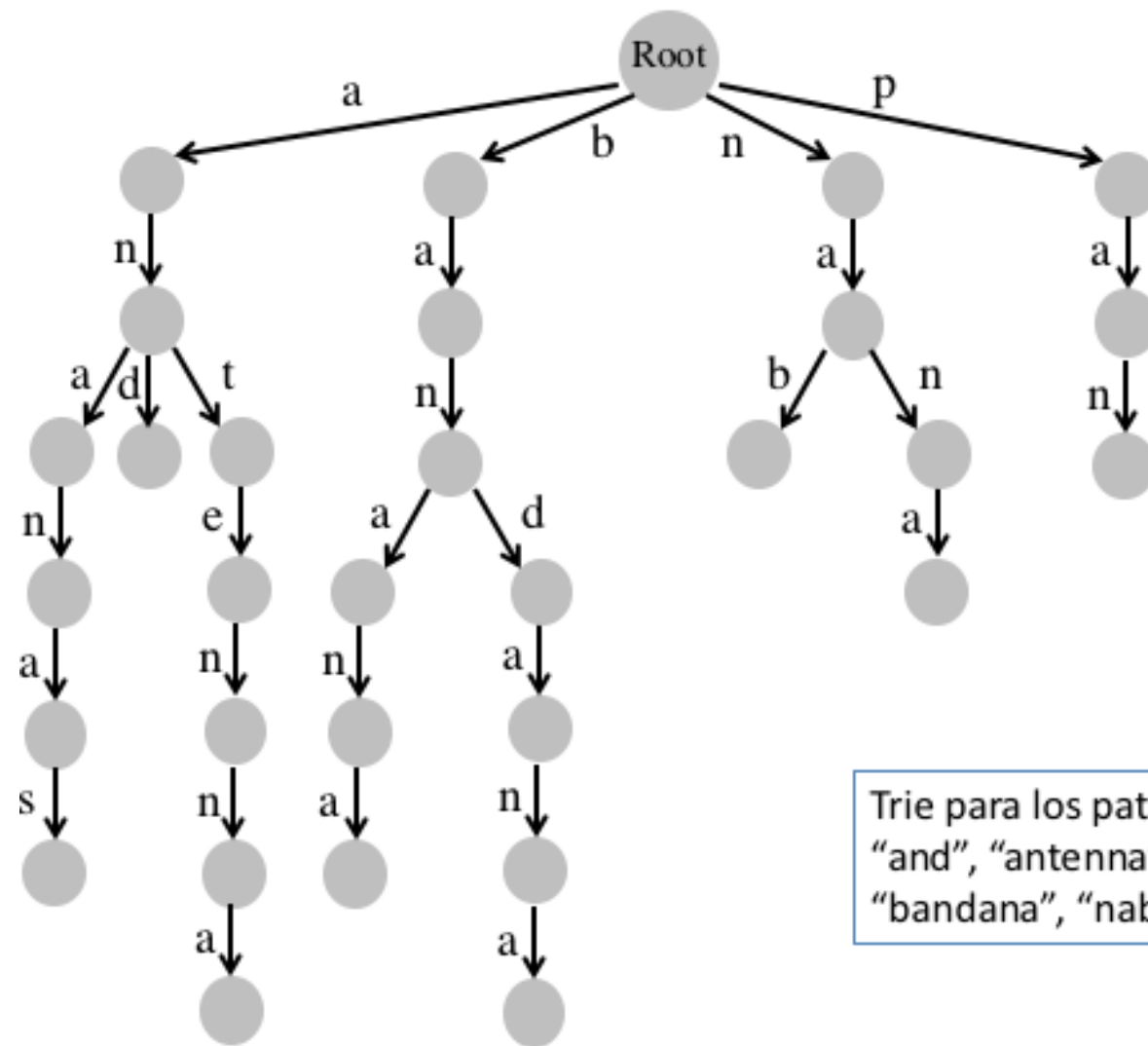
- En la sesión 2 implementamos búsquedas de patrones en secuencias cortas ($< 1K$)
 - Prueba `findMotif*` para buscar la DNA box de *E coli* TTATACACA en todo el genoma de *E coli*
 - $\rightarrow$ No es computacionalmente asumible
- Si queremos hacer búsquedas eficientes en genomas completos debemos recurrir a **nuevas estructuras de datos**

*Sesión 2, ejercicio 2

Complejidad computacional

- La solución de la sesión 2 es de **fuerza bruta**:
 - Para cada posición en un `Text`o, revisar si alguno de los `Patrones` buscados coincide
$$O(|\text{Text}| \cdot |\text{Patterns}|)$$
 - Con genomas, `|Text|` va a ser muy grande
 - Y a veces (p.e. alineamientos) `|Patterns|` también
 - Para un genoma humano, el genoma de referencia (`Text`) ocupa 3GB, y las lecturas de una secuenciación (`Patterns`) 1GB

Tries*

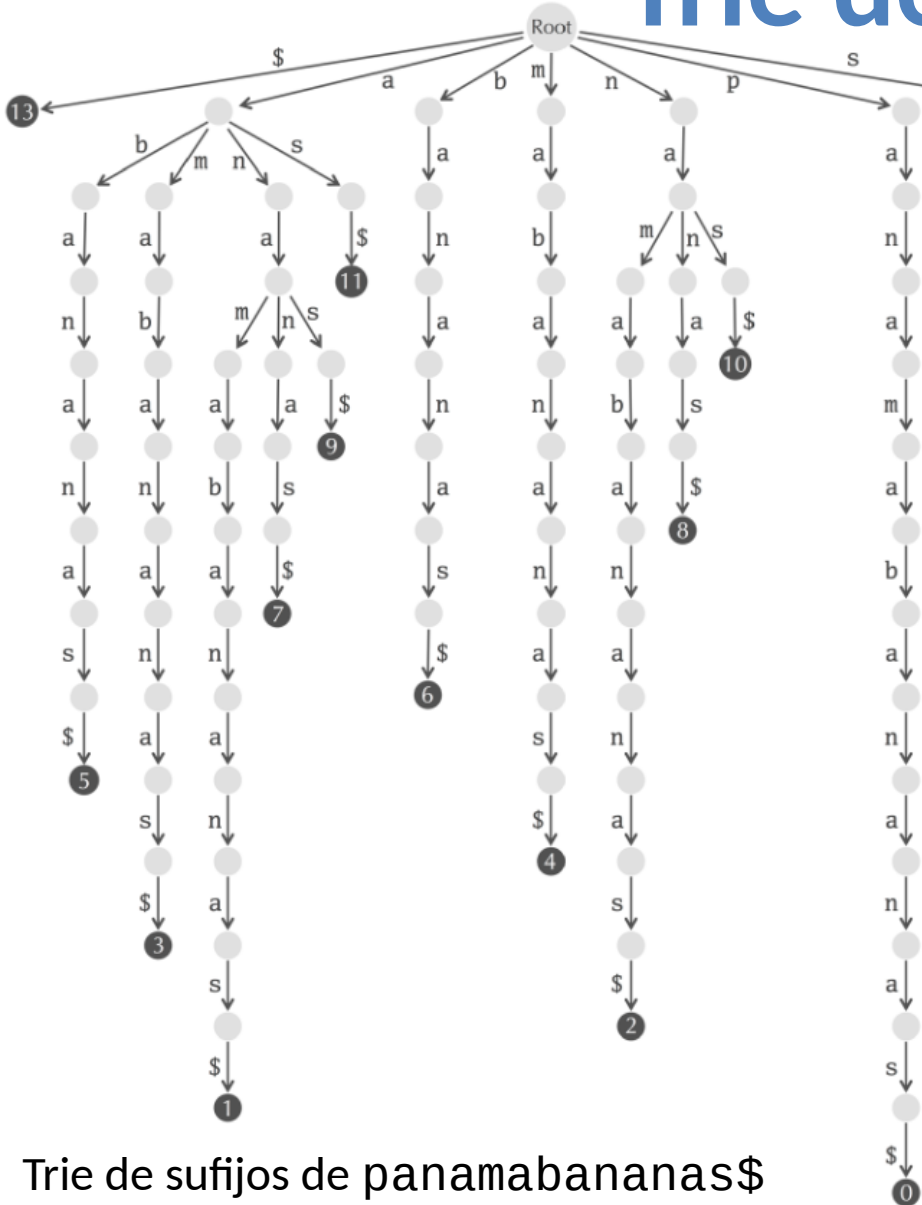


- **Objetivo:** agrupar todos los Patrones en una única estructura de datos
- **Búsqueda:** recorrer el árbol según Texto
- **Complejidad:**
 - $O(|\text{Texto}| \cdot |\text{PatrónMásLargo}|)$
- **Problemas:**
 - Construir el trie: $O(|\text{Patrones}|)$
 - Tamaño en memoria

Trie para los patrones "ananas",
"and", "antenna", "banana",
"bandana", "nab", "nana" y "pan"

*Este término tan raro fue acuñado por Edward Fredkin dos años después de que R. de la Briandais definiera esta estructura en 1959, y se refiere a la sílaba intermedia del verbo *retrieval* (recuperar)

Trie de sufijos

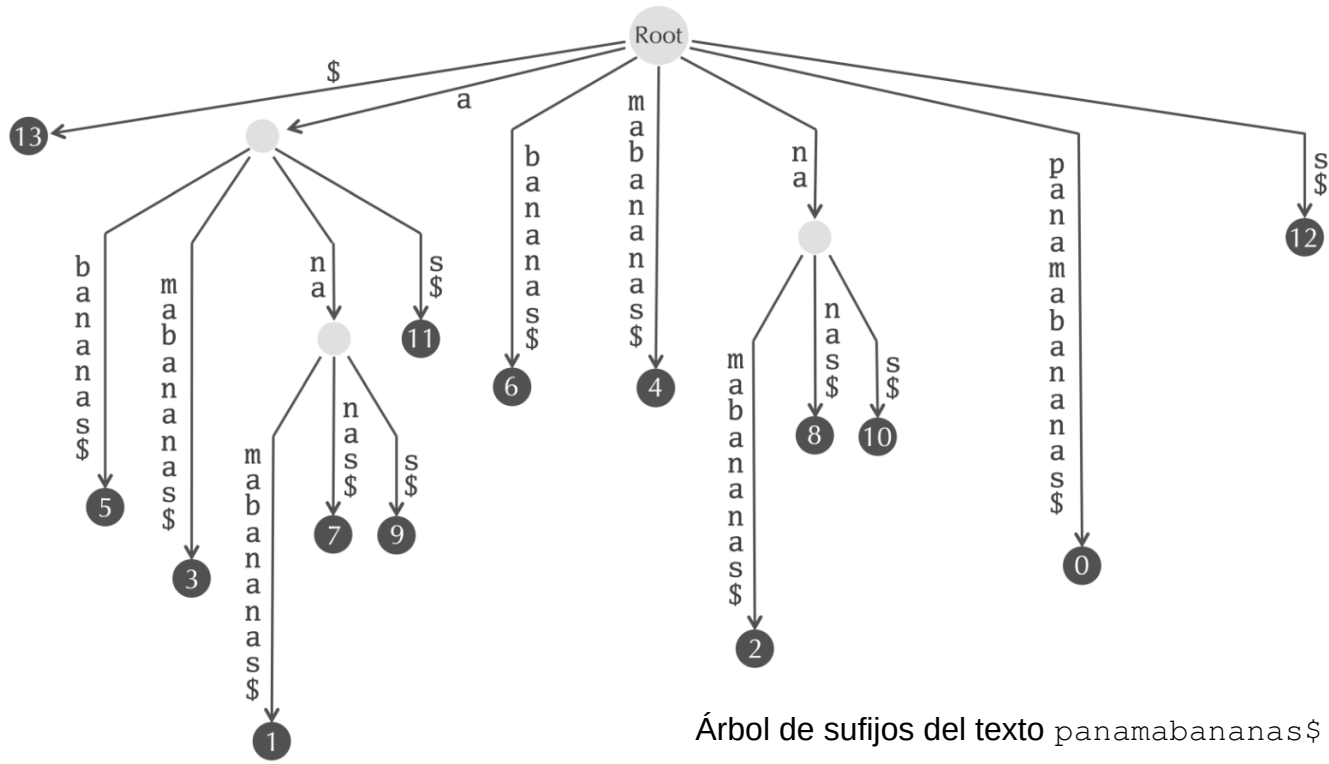


- En vez de hacer un trie de los Patrones lo hacemos de Texto
- **Construcción:**
 - $O(|\text{Texto}|^2)$
 - Más costoso, pero vale para cualquier búsqueda
- **Búsqueda:**
 - $O(|\text{SufijoÚnicoMásLargo}| \cdot |\text{Patrones}|)$
 - Mucho más eficiente si `Texto` es muy largo

Trie de sufijos de panamabananas\$

Árbol de sufijos

- Como el trie, pero fusionamos los nodos con sólo una rama de salida
 - Construcción más rápida: $O(|\text{Texto}|)$
 - Ocupa menos memoria



Árbol de sufijos del texto panamabananas\$

Array de sufijos

- Lista de las posiciones de todos los sufijos de un texto, ordenados alfabéticamente [Manber & Myers, 1989]

`SUFFIX_ARRAY("panamabananas$") =`
`[13, 5, 3, 1, 7, 9, 11, 6, 4, 2, 8, 10, 0, 12]`

- Mejora de memoria, aunque la complejidad sea igual que el árbol, $O(|\text{Text}|)$
 - La notación Big-O ignora los factores constantes, que son importantes con valores muy altos
 - $O(|\text{Text}|)$ es igual para $2 \cdot |\text{Text}|$ que para $1000 \cdot |\text{Text}|$

Sufijos Ordenados	Array de sufijos
\$	13
abananas\$	5
amabananas\$	3
anamabananas\$	1
ananas\$	7
anas\$	9
as\$	11
bananas\$	6
mabananas\$	4
namabananas\$	2
nanas\$	8
nas\$	10
panamabananas\$	0
s\$	12

Complejidad computacional

	Suffix Trie	Suffix Tree	Suffix Array
Memoria <i>(ej. h sapiens)</i>	$ Text ^2$ 1 TB	$ Text $ 60 GB	$ Text $ 12 GB
Construcción	$ Text ^2$	$ Text ^2$ <i>Existe algoritmo en $Text$</i>	$ Text \cdot \log(Text)$ <i>Existe algoritmo en $Text$</i>
Búsqueda	$ Text $	$ Text $	$ Text $

Ejercicio 1

- Implementar la función `suffixArray`:
 - **entrada:**
 - `text` (añadiendo un carácter especial `$` al final de la cadena)
 - **salida:** `suffixArray` correspondiente como una lista de posiciones
- Ejemplo:
 - `text`: "panamabananas\$"
 - `salida`: [13, 5, 3, 1, 7, 9, 11, 6, 4, 2, 8, 10, 0, 12]
- Prueba:
 - `text`='AACGATAGCGGTAGA\$'

BWT

- Estructuras de datos
- **Transformada de Burrows-Wheeler**
- Búsqueda de patrones

Transformada de Burrows-Wheeler

*definición y construcción
inversa y reconstrucción*

La transformada de Burrows-Wheeler

- Hemos conseguido búsquedas en tiempos lineales
- Hemos reducido mucho la memoria para las estructuras de datos adicionales necesarias

¿Podemos reducir más la memoria necesaria, hasta que sea parecida al tamaño del texto en el que hacemos las búsquedas?

Construcción

- **Matriz de Burrows-Wheeler (M)**
 - Se construye realizando todas las rotaciones cíclicas y ordenándolas alfabéticamente
- **Transformada de Burrows-Wheeler (BWT)**
 - Última columna de M

Rotaciones cíclicas

panamabananas\$
\$panamabananas
s\$panamabanana
as\$panamabanan
nas\$panamabana
anas\$panamaban
nanas\$panamaba
ananas\$panamab
bananas\$panama
abananas\$panam
mabananas\$pana
amabananas\$pan
namabananas\$pa
anamabananas\$p

M ("panamabananas\$")

\$panamabanana**s**
abananas\$pana**m**
amabananas\$pa**n**
anamabananas\$p**a**
ananas\$panama**b**
anas\$panamaba**n**
as\$panamabana**n**
bananas\$panam**a**
mabananas\$pan**a**
namabananas\$pa**a**
nanas\$panamab**a**
nas\$panamaban**a**
panamabananas\$
s\$panamabanana**a**

Ejercicio 2

- Implementar la función `bwt`:
 - **entrada**:
 - `text` (añadiendo un carácter especial `$` al final)
 - **salida**: transformada de Burrows-Wheeler
- Ejemplo:
 - `text`: "panamabananas\$"
 - `salida`: ['s', 'm', 'n', 'p', 'b', 'n', 'n', 'a', 'a', 'a', 'a', 'a', '\$', 'a']
- Prueba:
 - `text`='AACGATAGCGGTAGA\$'

De repeticiones a runs

- El efecto que tiene BWT sobre una cadena es transformar repeticiones en *runs* (cadenas de caracteres repetidos de manera consecutiva*):
 - `bwt( "panamabananas$" ) = smnpbnnaaaaa$a`
- ¿Por qué?
- Muy bonito, pero ¿en su construcción no necesitamos mantener una matriz $| \text{Text} | \cdot | \text{Text} |$?
- ¿Podemos reconstruir la cadena original a partir de su BWT? Si no, no nos sirve de nada

*Este es un método usado para compresión, por ejemplo, en bzip2

Inversa de BWT

- Sea un texto desconocido cuya BWT es
 - `bwt(texto)=ard$rcaaaabb`
- Conocemos la primera y última columna de M:
 - La última es
 - `bwt(texto)`
 - La primera es
 - `sort(bwt(texto))`

[illegible]

Inversa de BWT

- El primer símbolo de la cadena debe seguir al \$ en todas las rotaciones
 - Basta con mirar la fila donde \$ está en la última columna:
 - El primer símbolo de la cadena será el que esté en la primera columna de dicha fila

\$	a	?	?	?	?	?	?	?	?	?	a
a	?	?	?	?	?	?	?	?	?	?	r
a	?	?	?	?	?	?	?	?	?	?	d
a	?	?	?	?	?	?	?	?	?	?	\$
a	?	?	?	?	?	?	?	?	?	?	r
a	?	?	?	?	?	?	?	?	?	?	c
b	?	?	?	?	?	?	?	?	?	?	a
b	?	?	?	?	?	?	?	?	?	?	a
c	?	?	?	?	?	?	?	?	?	?	a
d	?	?	?	?	?	?	?	?	?	?	a
r	?	?	?	?	?	?	?	?	?	?	b
r	?	?	?	?	?	?	?	?	?	?	b

Inversa de BWT

- Siguiendo la misma lógica, a la a le puede seguir una b, c ó d

\$	a	b	?	?	?	?	?	?	?	?	a
a	?	?	?	?	?	?	?	?	?	?	r
a	?	?	?	?	?	?	?	?	?	?	d
a	?	?	?	?	?	?	?	?	?	?	\$
a	?	?	?	?	?	?	?	?	?	?	r
a	?	?	?	?	?	?	?	?	?	?	c
b	?	?	?	?	?	?	?	?	?	?	a
b	?	?	?	?	?	?	?	?	?	?	a
c	?	?	?	?	?	?	?	?	?	?	a
d	?	?	?	?	?	?	?	?	?	?	a
r	?	?	?	?	?	?	?	?	?	?	b
r	?	?	?	?	?	?	?	?	?	?	b

\$	a	c	?	?	?	?	?	?	?	?	a
a	?	?	?	?	?	?	?	?	?	?	r
a	?	?	?	?	?	?	?	?	?	?	d
a	?	?	?	?	?	?	?	?	?	?	\$
a	?	?	?	?	?	?	?	?	?	?	r
a	?	?	?	?	?	?	?	?	?	?	c
b	?	?	?	?	?	?	?	?	?	?	a
b	?	?	?	?	?	?	?	?	?	?	a
c	?	?	?	?	?	?	?	?	?	?	a
d	?	?	?	?	?	?	?	?	?	?	a
r	?	?	?	?	?	?	?	?	?	?	b
r	?	?	?	?	?	?	?	?	?	?	b

\$	a	d	?	?	?	?	?	?	?	?	a
a	?	?	?	?	?	?	?	?	?	?	r
a	?	?	?	?	?	?	?	?	?	?	d
a	?	?	?	?	?	?	?	?	?	?	\$
a	?	?	?	?	?	?	?	?	?	?	r
a	?	?	?	?	?	?	?	?	?	?	c
b	?	?	?	?	?	?	?	?	?	?	a
b	?	?	?	?	?	?	?	?	?	?	a
c	?	?	?	?	?	?	?	?	?	?	a
d	?	?	?	?	?	?	?	?	?	?	a
r	?	?	?	?	?	?	?	?	?	?	b
r	?	?	?	?	?	?	?	?	?	?	b

Propiedad principio-fin de M

La k-ésima ocurrencia de un símbolo en la primera columna y su k-ésima ocurrencia en la última columna corresponden a la misma posición en el texto original

$\$$ ₁	a	b	?	?	?	?	?	?	?	a ₁
a ₁	?	?	?	?	?	?	?	?	?	r ₁
a ₂	?	?	?	?	?	?	?	?	?	d ₁
a ₃	?	?	?	?	?	?	?	?	?	$\$$ ₁
a ₄	?	?	?	?	?	?	?	?	?	r ₂
a ₅	?	?	?	?	?	?	?	?	?	c ₁
b ₁	?	?	?	?	?	?	?	?	?	a ₂
b ₂	?	?	?	?	?	?	?	?	?	a ₃
c ₁	?	?	?	?	?	?	?	?	?	a ₄
d ₁	?	?	?	?	?	?	?	?	?	a ₅
r ₁	?	?	?	?	?	?	?	?	?	b ₁
r ₂	?	?	?	?	?	?	?	?	?	b ₂

Para la inversa de BWT, esta propiedad nos permite decidir cuál de las anteriores es la opción correcta!

Inversa de BWT

- Continuamos con el proceso hasta resolver la primera fila de M, que contiene el texto original

$\$_1$	a	b	r	a	c	a	d	a	b	r	a_1
a_1	?	?	?	?	?	?	?	?	?	?	r_1
a_2	?	?	?	?	?	?	?	?	?	?	d_1
a_3	?	?	?	?	?	?	?	?	?	?	$\$_1$
a_4	?	?	?	?	?	?	?	?	?	?	r_2
a_5	?	?	?	?	?	?	?	?	?	?	c_1
b_1	?	?	?	?	?	?	?	?	?	?	a_2
b_2	?	?	?	?	?	?	?	?	?	?	a_3
c_1	?	?	?	?	?	?	?	?	?	?	a_4
d_1	?	?	?	?	?	?	?	?	?	?	a_5
r_1	?	?	?	?	?	?	?	?	?	?	b_1
r_2	?	?	?	?	?	?	?	?	?	?	b_2

Ejercicio 3

- Implementar la función `ibwt*`:
 - **entrada:**
 - `transformada` (con un único símbolo `$`)
 - **salida:** `texto` tal que `bwt(texto)=transformada`
- Ejemplo:
 - `transformada`: `ard$rcaaaaabb`
 - `salida`: `abracadabra$`
- Prueba:
 - `transformada`=`'enwvpeoseu$llt'`

* Se recomienda programar previamente una función `subscripts` que, dada una cadena de texto, retorne el número de ocurrencia de cada letra, para hacer uso de la propiedad principio-fin

BWT

- Estructuras de datos
Transformada de Burrows-Wheeler
- **Búsqueda de patrones**
- Búsquedas aproximadas

Búsqueda de patrones

búsqueda

mejoras en rendimiento y memoria

Búsqueda de patrones con BWT

- Las coincidencias de un patrón tienden a agruparse juntas en la matriz de BWT:

\$panamabananas
abananas\$panam
amabananas\$pan
anamabananas\$p
ananas\$panamab
anas\$panamaban
as\$panamabanan
bananas\$panama
mabananas\$pana
namabananas\$pa
nanas\$panamaba
nas\$panamabana
panamabananas\$
s\$panamabanan

Podríamos ir buscando las filas comenzando con el primer símbolo del patrón buscado

Luego, para esas filas buscamos cuáles contienen en su segunda columna el segundo carácter del patrón. Y así sucesivamente.

En el ejemplo, el patrón **ana**

Memoria

- La aproximación para búsqueda de patrones no puede confiar en almacenar una matriz M de tamaño $|Text| \times |Text|$
 - Debemos diseñar una búsqueda que sólo utilice la primera y última columna de M
 - Para ello usaremos de nuevo la propiedad principio-fin de la matriz M

Búsqueda de patrones con BWT (ii)

- patrón=ana, texto="panamabananas\$"

\$ ₁	panamabanana	s ₁
a ₁	bananas\$pana	m ₁
a ₂	mabananas\$pa	n ₁
a ₃	namabananas\$	p ₁
a ₄	nanas\$panama	b ₁
a ₅	nas\$panamaba	n ₂
a ₆	s\$panamabana	n ₃
b ₁	ananas\$panam	a ₁
m ₁	abananas\$pan	a ₂
n ₁	amabananas\$p	a ₃
n ₂	anas\$panamab	a ₄
n ₃	as\$panamaban	a ₅
p ₁	anamabananas	\$ ₁
s ₁	\$panamabanan	a ₆

1) Empezamos por la última letra (a) y buscamos todas las filas que la tengan en la primera columna

\$ ₁	panamabanana	s ₁
a ₁	bananas\$pana	m ₁
a ₂	mabananas\$pa	n ₁
a ₃	namabananas\$	p ₁
a ₄	nanas\$panama	b ₁
a ₅	nas\$panamaba	n ₂
a ₆	s\$panamabana	n ₃
b ₁	ananas\$panam	a ₁
m ₁	abananas\$pan	a ₂
n ₁	amabananas\$p	a ₃
n ₂	anas\$panamab	a ₄
n ₃	as\$panamaban	a ₅
p ₁	anamabananas	\$ ₁
s ₁	\$panamabanan	a ₆

2) Nos quedamos con aquellas que en la última columna tengan la penúltima letra (n)

Búsqueda de patrones con BWT (ii)

- patrón=ana, texto="panamabananas\$"

\$ ₁	panamabanana	s ₁
a ₁	bananas\$pana	m ₁
a ₂	mabananas\$pa	n ₁
a ₃	namabananas\$	p ₁
a ₄	nanas\$panama	b ₁
a ₅	nas\$panamaba	n ₂
a ₆	s\$panamabana	n ₃
b ₁	ananas\$panam	a ₁
m ₁	abananas\$pan	a ₂
n₁	a mabananas\$p	a₃
n₂	a nas\$panamab	a₄
n₃	a s\$panamaban	a₅
p ₁	anamabananas	\$ ₁
s ₁	\$panamabanan	a ₆

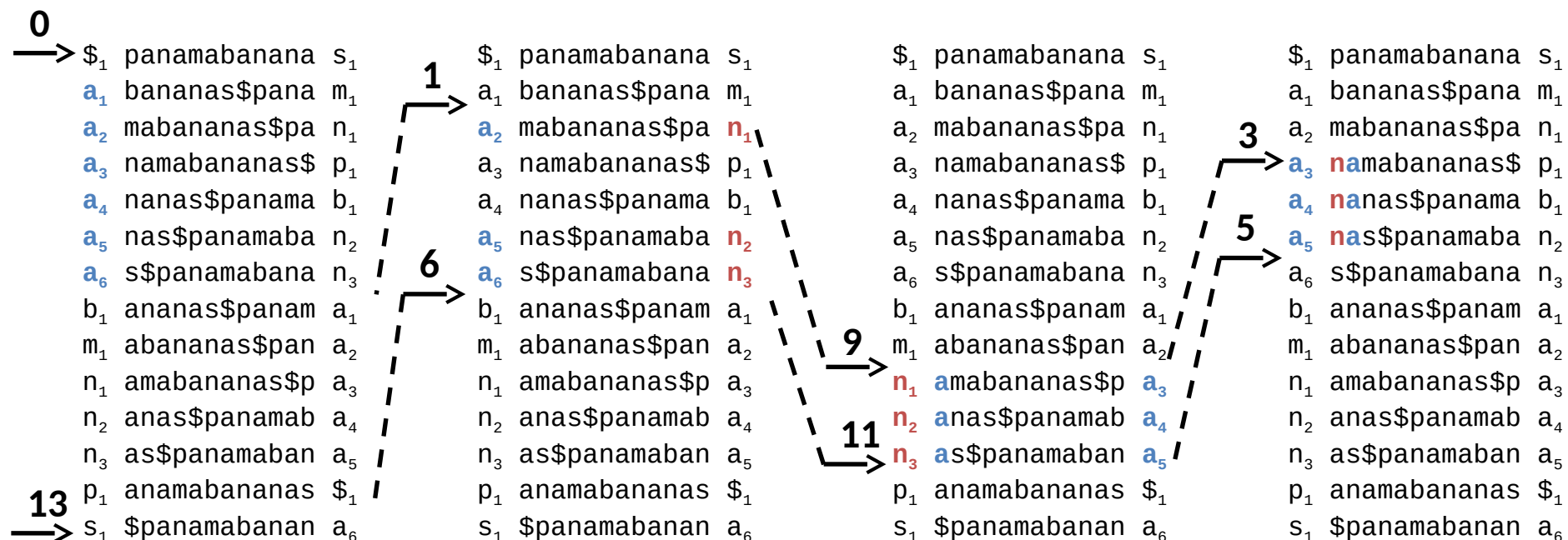
3) Identificamos las coincidencias anteriores en la primera columna y repetimos el proceso

\$ ₁	panamabanana	s ₁
a ₁	bananas\$pana	m ₁
a ₂	mabananas\$pa	n ₁
a₃	n a m abananas\$	p ₁
a₄	n a n as\$panama	b ₁
a₅	n a s \$panamaba	n ₂
a ₆	s\$panamabana	n ₃
b ₁	ananas\$panam	a ₁
m ₁	abananas\$pan	a ₂
n ₁	a m abananas\$p	a₃
n ₂	a n as\$panamab	a₄
n ₃	a s \$panamaban	a₅
p ₁	anamabananas	\$ ₁
s ₁	\$panamabanan	a ₆

4) Al terminarse el patrón, la posición de los caracteres en la columna primera nos da las ocurrencias del patrón en el texto

Búsqueda de patrones con BWT (ii)

- Este modo de búsqueda se puede ver como una actualización de los punteros que indican las coincidencias en tramos crecientes del patrón:



Mapeo primero-a-último

- Traducción de la primera y última columna a punteros numéricos de fila
 - “ s_1 está en la fila 0 de la última columna $\rightarrow s_1$ está en la fila 13 de la primera columna”

i	Primera Columna	Última Columna	PrimeroAUltimo(i)
0	s_1	s_1	13
1	a_1	m_1	8
2	a_2	n_1	9
3	a_3	p_1	12
4	a_4	b_1	7
5	a_5	n_2	10
6	a_6	n_3	11
7	b_1	a_1	1
8	m_1	a_2	2
9	n_1	a_3	3
10	n_2	a_4	4
11	n_3	a_5	5
12	p_1	s_1	0
13	s_1	a_6	6

BWMatching

```
bwMatching(últimaColumna, patrón, primeroAÚltimo)
  top <- 0
  bottom <- |últimaColumna|
  mientras top <= bottom
    si patrón no vacío
      símbolo <- última letra en patrón
      eliminar última letra de patrón
      posiciones <- [top:bottom]
      si últimaColumna[posiciones] contiene símbolo
        topi <- primera posición de símbolo en dichas posiciones
        bottomi <- ultima posición de símbolo en dichas posiciones
        top <- primeroAÚltimo[topi]
        bottom <- primeroAÚltimo[bottomi]
      si no
        return 0
    si no
      return posiciones de bottom a top
  return 0
```

Ejercicio 4

- Implementar la función `bwMatching` según el pseudocódigo anterior
 - **entrada:**
 - `ultimaColumna` de una matriz M correspondiente al la BWT del texto en el que tenemos que buscar el patrón
 - `primeroAUltimo` lista de punteros en la matriz M
 - [Opcionalmente, se pueden sustituir estos dos parámetros de entrada por el texto, y construir la matriz y la lista de punteros dentro de la función]
 - `patron` que se busca
 - **salida:** `posiciones` de las filas de la matriz M que contienen el patrón
- Ejemplo:
 - texto: `panamabananas$`
 - patrón: `ana`
 - salida: `[3, 4, 5]`
- Prueba:
 - `texto=oricVC*` (con un `$` al final)
 - patrón=`cgga`

Ejercicio

- Carga el genoma de *V cholerae* (~1.1 Mbases)
 - Aplica el bwMatching a sus primeras 10 Kbases, monitorizando o estimando:
 - Tiempo invertido en construir M
 - Memoria requerida para almacenar M
 - Tiempo invertido en la búsqueda
 - Repite con 50 Kbases
- ¿Este método de búsqueda es escalable hasta 1.1 Mbases?
 - ¿Cuál es el principal cuello de botella?
 - Uno se puede desanimar viendo que un navegador o editor de texto cualquiera nos hace la búsqueda sin problemas para 1.1M a este nivel... ¡paciencia! :)

Mejora 1) Recuperando la posición de los patrones

- Antes de meternos con la reducción de memoria, nos falta un detalle:
 - ¡No sabemos dónde están los patrones encontrados en el texto!
 - Sólo estamos retornando las posiciones en la matriz
- Necesitaremos un array de sufijos del texto
 - Retornar `suffixArray[top:bottom+1]` en vez de las posiciones de `bottom` a `top`

Mejora 2) Reduciendo la memoria

- Estamos reservando memoria para M
 - Si $|\text{texto}|=10^6 \rightarrow |M|=|\text{texto}|^2=10^{12}$
 - Casi 1000 GB para un genoma del tamaño de *V cholerae*
- Realmente sólo necesitamos $|\text{pattern}|$ columnas de M para asegurar la búsqueda
 - Si $|\text{texto}|=10^6$ y $|\text{pattern}|=10 \rightarrow 10^7$
 - 10 M para un genoma del tamaño de *V cholerae*
 - Y podemos buscar k-meros de hasta tamaño 10
- Lo mismo pasa con la construcción de `suffixArray`
 - Podemos modificar la versión del ej 1

Mejora 2) Reduciendo la memoria

1) Rotar

pana|\$

\$pan|s

s\$pa|a

as\$|n

nas\$|a

anas|n

nana|a

anan|b

bana|a

aban|m

maba|a

amab|n

nama|a

anam|p

2) Ordenar

\$pans

abanm

amabn

anamp

ananb

anasn

as\$pn

banaa

mabaa

namaa

nanaa

nas\$a

pana\$

s\$paa

Ejemplo: transformada de
panamabananas\$ si queremos
buscar patrones de longitud máxima 4

Mejora 3) Primeras ocurrencias

- La memoria es claramente un problema
 - Para un genoma de humano (~3GB) tan sólo mantener la primera y/o última columna de M en RAM es bastante costoso
- Realmente, no necesitamos toda la primera columna, sólo el índice de la primera ocurrencia de cada símbolo
 - 3GB → 5 caracteres

Primeras ocurrencias

1) Rotar

pana|\$

\$pan|s

s\$pa|a

as\$p|n

nas\$|a

anas|n

nana|a

anan|b

bana|a

aban|m

maba|a

amab|n

nama|a

anam|p

2) Ordenar

\$pans

abanm

amabn

anamp

ananb

anasn

as\$pn

banaa

mabaa

namaa

nanaa

nas\$a

pana\$

s\$paa

3) Primeras oc

\$ 0

a 1

b 7

m 8

n 9

p 12

Ejercicio 5

- Implementar la función `firstOccurrence*`
 - **entrada:**
 - `symbol` carácter del que queremos contar el número de ocurrencias en texto
 - `text` texto en el que queremos contar el símbolo
 - **salida:** primera posición de `symbol` en `text`
- Ejemplo:
 - texto: `$aaaaaabmnnps`
 - símbolo: `n`
 - salida: 9
- Prueba:
 - Aplicar a todos los símbolos en `$aaaaaabmnnps` para obtener una lista de primeras ocurrencias
 - PISTA: Muy sencillo utilizando `map`

*PISTA: trivial en python usando la función `index` para `str`

lambda

- Una forma compacta y elegante de programar en python es el uso de la función lambda
- Es una forma abreviada de definir una función
 - `lambda x:x+3` $\leftrightarrow$ `def(x):`
`return x+3`
- Lambda facilita la programación funcional en Python
 - Es decir, permite incluir funciones como argumentos de otras funciones

map

- map es una función que aplica otra función f a todos los elementos de una lista s

```
map( lambda x: x*2+10, [2, 3, 4] )  
→ [14, 16, 18]
```

- Otro uso común equivalente a map:

```
[x*2+10 for x in [2, 3, 4]]
```

filter

- `filter` selecciona sólo los valores de una lista S que cumplan la condición definida en una función f

```
filter(lambda x:x>10, [2, 8, 13, 33])  
→ [13, 33]
```


reduce

- `reduce` es una función que aplica una función f a los dos primeros elementos de una lista s y continúa aplicando f al resultado más el siguiente elemento de s

`reduce(lambda x, y: x+y+1, [2, 3, 4])`
`→ 11`

NOTA: desaconsejado en Python 3 por poco legible.
Relegado a `functools.reduce()`

Ejercicio

- Implementar la función `count`
 - **entrada:**
 - `symbol` carácter del que queremos contar el número de ocurrencias en `texto`
 - `n` longitud en `texto` hasta el que queremos contar ocurrencias
 - `text` texto en el que queremos contar el símbolo
 - **salida:** número de veces que aparece `symbol` en `text[:n]`
- Ejemplo:
 - texto: `smnpbnnaaaaa$a`
 - símbolo: `n`
 - `n`: 10
 - salida: 3

Ejercicio

- Construir `bwMatchingV2` modificando la sección de pseudocódigo marcada en verde en `bwMatching` por:

```
top <- firstOccurrence(símbolo) +  
      count(símbolo, top, últimaColumna)  
bottom <- firstOccurrence(símbolo) +  
          count(símbolo, bottom+1, últimaColumna) - 1
```

- Con esta modificación eliminamos el mapeo primero-a-último y primeraColumna de la búsqueda
 - Aunque las sustituimos por `firstOccurrence` y `count`
- Estudiar si se ha obtenido alguna mejora en velocidad o memoria con respecto a la versión anterior
 - ¿Por qué?

Mejora 4) La pregunta del millón

- Cuando empezamos a trabajar con cadenas de 1 millón de elementos o más, empieza a haber problemas a distintos niveles:
 - `range` estará generando una lista también de 1M
 - Utilizar `xrange` para iterar en los bucles*
 - las listas de python no son eficientes
 - Utilizar `numpy.array`
 - Envoltorio de estructuras directamente de C

*No es necesario en Python 3, donde `range` ya funciona como `xrange`

numpy.array

- En **Python** tenemos la clase `list`
 - No es exactamente un array, pues tiene un tamaño dinámico y puede contener elementos de distintos tipos
 - Esto la hace muy flexible, pero menos eficiente
- La biblioteca **numpy** ofrece `array`
 - Hay que declarar su tamaño y su tipo
 - Aún así, muy flexible en la declaración de tipos
 - Permite arrays multidimensionales (matrices)

numpy.array

```
import numpy as np
a=np.array([1,2,3,4]) #declaración con valores
a=np.empty(2000,dtype="i4") #2000 enteros de 4 bytes (ligeramente + rápido)
a=np.zeros(2000,dtype="i4") #2000 enteros de 4 bytes iniciados a cero
#Tabla de 100 filas y dos columnas, una de cadenas de 10 caracteres llamada
# 'name' y otra de enteros de 1 byte llamada 'age'
t=np.empty(100,dtype=[('name','S10'),('age','i1')])
t[12]['name']    #acceso a un elemento
t[0]             #acceso a una fila
t['name']        #acceso a una columna

t[0]=['Jose', 33]      #modificación de fila o elementos
t[25]['name']='Elisa'
```

El uso es similar al de una lista, salvo por las restricciones al tamaño. Permite definir tipos distintos en las columnas de la tabla/matriz mediante el argumento dtype. Los tipos se pueden modular en tamaño con una sintaxis simple. Podemos acceder a filas o columnas también de manera sencilla.

Ejercicio

- Con lo que hemos visto deberíamos poder tratar con un genoma pequeño como el de *V cholerae* (1.1 Mbases) con:
 - Tiempo de búsqueda < 1s
 - Tiempo de construcción de estructuras < 10s
 - Memoria RAM utilizada despreciable
- Revisa tu solución intentando llegar a este punto

(mi) ejercicio

- Estos son mis resultados con las distintas versiones de la búsqueda
 - El bloqueo de cada versión suele venir dado por el consumo de RAM

Versión	Alcance	Mejoras
3	10K	Versión con <code>firstOccurrence</code> y <code>suffixArray</code>
4	100K	Reducir la longitud de secuencia guardada en la búsqueda
5	500K	Reducir la longitud también en BWT
6	1.1M <i>V chol</i>	Reducir también en funciones <code>count</code> y <code>checkpoints</code> ~3s en la construcción, <1s en búsquedas
7	4.6M <i>E coli</i>	Aplicar <code>xrange</code> y <code>np.array</code> a todas las funciones ~15s en la construcción, <1s en búsquedas

Recapitulando

- Para hacer búsquedas rápidas en genomas, exploramos estructuras de datos nuevas
- La mejora en tiempo va asociada a un incremento de memoria para almacenar dichas estructuras
 - Mejoramos las estructuras (tree → trie → array)
 - 1TB → 60GB → 12 GB para el genoma humano (3GB)
 - Seguimos mejorando (BWT, firstOccurrence)
 - Con un poco más de ingenio, llegaremos hasta unos 4.5GB en RAM para el genoma humano.

Mejora 5) Arrays parciales* de sufijos

- Añadiendo arrays de sufijos a `BWmatching` estamos aumentando la memoria de nuevo, cuando justo lo que necesitamos es reducirla
- `SuffixArrayk(texto)` contiene sólo los valores de `SuffixArray(texto)` que sean múltiplos de k
 - Típicamente $k=100$

*Esta es una aproximación usual cuando un algoritmo ya no da más de sí: precomputar ciertos valores muy usados intentando minimizar el impacto en memoria.

Mejora 6) Puntos de control en BW

- La función `count` tiene una complejidad $O(|\text{Text}|)$ y se ejecuta con cada rechequeo de `top` y `bottom`
- Podríamos precomputarlo, pero de nuevo empeoraríamos el consumo de memoria
- **Checkpoints:** Como en el caso de los arrays de sufijos parciales, la solución es precomputar algunos `count` e inferir el resto cuando sea necesario

- Los checkpoints para “panamabananas\$”
 - El resultado de aplicar count a cada símbolo k y posición i
- Sólo almacenamos las posiciones múltiplo de $C=5$
 - Lo normal, $C=100$
 - En el dibujo, en negrita
- $\text{count}(a, 13)$
 - Vamos al checkpoint más cercano, donde $a=3$
 - Avanzo por *últimaColumna* hasta llegar al 13, viendo si hay más aes en la posición **10** (sí), **11** (sí) y **12** (no)
 - En total:
 - $\text{count}(a, 13)=3+2=5$

i	<i>últimaColumna</i>		count						
		\$	a	b	m	n	p	s	
0	s	0	0	0	0	0	0	0	
1	m	0	0	0	0	0	0	1	
2	n	0	0	0	1	0	0	1	
3	p	0	0	0	1	1	0	1	
4	b	0	0	0	1	1	1	1	
5	n	0	0	1	1	1	1	1	
6	n	0	0	1	1	2	1	1	
7	a	0	0	1	1	3	1	1	
8	a	0	1	1	1	3	1	1	
9	a	0	2	1	1	3	1	1	
10	a	0	3	1	1	3	1	1	
11	a	0	4	1	1	3	1	1	
12	\$	0	5	1	1	3	1	1	
13	a	0	5	1	1	3	1	1	
		1	6	1	1	3	1	1	

Ejercicio

- Implementar la función `checkpoints`
 - **entrada:**
 - `lastColumn` transformada de Burrows-Wheeler de un texto
 - `symbols` todos los símbolos del texto (sólo una vez cada uno)
 - `c` determina cada cuantas posiciones de `lastColumn` establecemos un checkpoint
 - **salida:** vectores con los puntos de chequeo para cada símbolo
- Ejemplo:
 - texto: `smnpbnnaaaaa$a`
 - símbolos: `smnpba$`
 - `c`: 5
 - salida: `{'$': [0, 0, 0], 'a': [0, 0, 3], 'b': [0, 1, 1], 'm': [0, 1, 1], 'n': [0, 1, 3], 'p': [0, 1, 1], 's': [0, 1, 1]}`

Ejercicio

- Implementar la función `count s`
 - **entrada:**
 - `symbol` carácter del que queremos contar el número de ocurrencias en texto
 - `pos` longitud en texto hasta el que queremos contar ocurrencias
 - `text` transformada de Burrows-Wheeler (i.e. últimaColumna)
 - `checkpoints` vectores calculados cada `k` posiciones para todos los símbolos
 - `k`
 - **salida:** número de veces que aparece `symbol` en `text[:pos]`
- Ejemplo:
 - texto: `smnpbnnaaaaa$a`
 - símbolo: `a`
 - pos: 13
 - checkpoints: `checkpoints(smpbnnaaaaa$a, smnpba$, 5)`
 - k: 5
 - salida: 5

Ejercicio

- Prueba tu método con las mejoras que hayas implementado en genomas crecientes en tamaño
 - Aquí están los tiempos que yo consigo:

Especie	Tamaño	Construcción ¹	Búsqueda ¹
<i>V cholerae</i>	1.1 M	3s	0.03s
<i>E coli</i>	4.6 M	15s	0.12s
<i>B brevis</i>	6.3 M	24s	0.2s
<i>H sapiens</i> (chr. Y)*	26.4 M	107s ²	0.2s
<i>H sapiens</i> (chr. 1)*	503.0 M		

¹Tiempos para buscar un 9-mer, y construyendo para encontrar 9-mers como mucho. El espacio en RAM es despreciable hasta llegar a H sapiens, con 0.8G de RAM para el chrY

²25s rotación cíclica, 45s ordenación, 4s en firstOccurrences, 33s en checkpoints.

*Descargable aquí: <http://hgdownload.cse.ucsc.edu/downloads.html#human>

BWT

- Estructuras de datos
- Transformada de Burrows-Wheeler
- Búsqueda de patrones
- **Búsquedas aproximadas**

Búsquedas aproximadas

arrays parciales de sufijos

puntos de control en BWT

Búsqueda de patrones aproximados

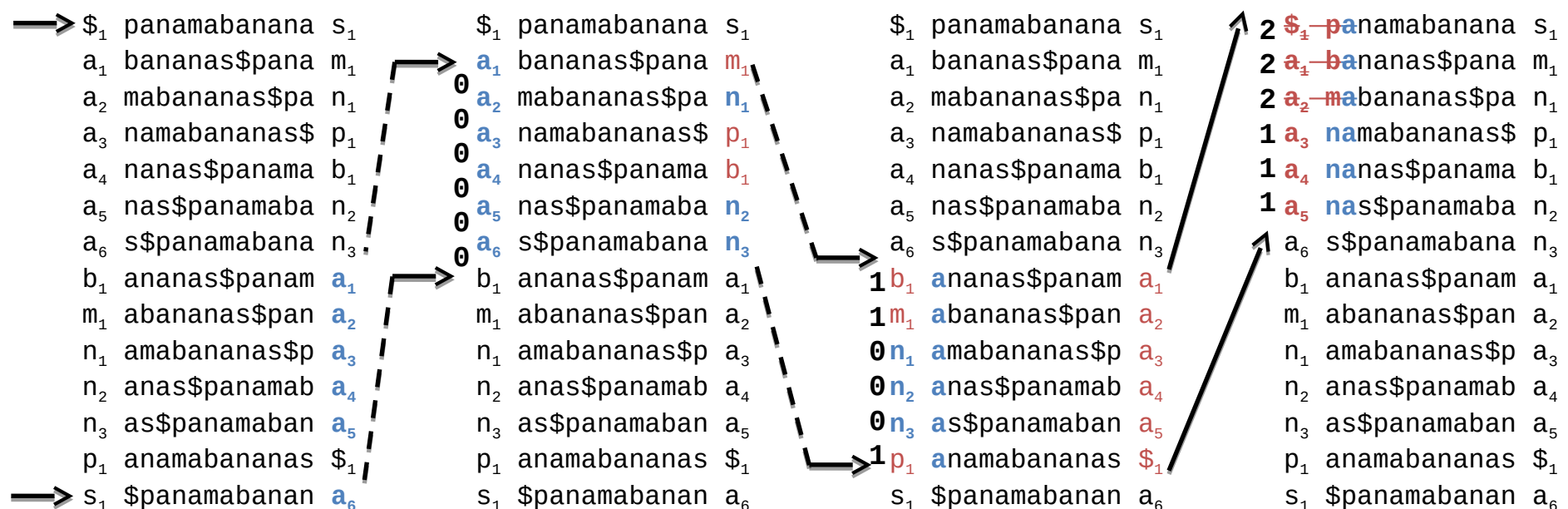
- En general, vamos a buscar patrones que aparezcan con como mucho d discordancias
- **Teorema:** si dos cadenas de longitud n coinciden con como mucho d discordancias, entonces comparten un k -mero de longitud $k=n/(d+1)$
 - Prueba: el modo más disperso de las discordancias es tener una cada $k=n/(d+1)$ posiciones → entre dos de dichas posiciones no habrá discordancia

Búsqueda de patrones aproximados

- Gracias a este teorema podemos utilizar los métodos rápidos de búsqueda exacta de patrones para búsquedas con discordancias:
 1. Dividir `pattern` en $k=n/(d+1)$ partes (**semillas**)
 2. Búsqueda exacta de semillas en `texto` (**detección de semillas**)
 3. Extender las semillas en ambas direcciones para verificar si el patrón ocurre con un máximo de d discordancias (**extensión de semillas**)

Búsqueda aproximada y BWT

- BWT matching pero con un array de discordancia, por ejemplo:
 - text=panamabananas\$, pattern=dna, d=1



Búsqueda aproximada y rendimiento

- La búsqueda aproximada puede reducir mucho el rendimiento
 - Se consideran muchas cadenas candidatas
 - d se suele mantener < 3

Especie	Tamaño	d=0 ¹	#	d=1	#	d=2	#	d=3	#
<i>V cholerae</i>	1.1 M	0.03	1	0.4	61	5.2	984	36.4	8579
		0.03	0	0.07	0	0.1	0	0.6	1

¹Tiempos en segundos para buscar el 9-mer **CGGAACGGT**, con hasta d inconsistencias y # de veces que se encuentra el patrón. También para el 20-mer CTATTCGCCGCCAACGTGGG

Ejercicio

- Implementar la función `bwMatchingV3`
 - **entrada:**
 - `text` texto en el que queremos encontrar el patrón
 - `pattern` patrón a buscar
 - `d` número de mutaciones sobre el patrón permitidas
 - `cf` transformada de Burrows Wheeler (primera columna de la matriz) de `text`*
 - `fo` primeras ocurrencias en la primera columna de las rotaciones cíclicas de `text`**
 - `sa` array de sufijos de `text`
 - `cp` checkpoints de la transformada ($k=5$)
 - **salida:** lista con todas las posiciones de `pattern` con hasta `d` mutaciones en `text`

*Todos estos parámetros adicionales son para evitar tener que construir la estructura de datos de la transformada con cada nueva búsqueda sobre `text`

** Ver ejercicio 5

Ejercicio

- Ejemplo:
 - text: panamabananas\$
 - pattern: bna
 - d: 1
 - cf: smnpbnnaaaaa\$a
 - fo: {'a': 1, 'b': 7, '\$': 0, 'm': 8, 'n': 9, 'p': 12, 's': 13}
 - sa: [13, 5, 3, 1, 7, 9, 11, 6, 4, 2, 8, 10, 0, 12]
 - cp: {'a': [0, 0, 3], 'b': [0, 1, 1], '\$': [0, 0, 0], 'm': [0, 1, 1], 'n': [0, 1, 3], 'p': [0, 1, 1], 's': [0, 1, 1]}
 - salida: [1, 9, 7]

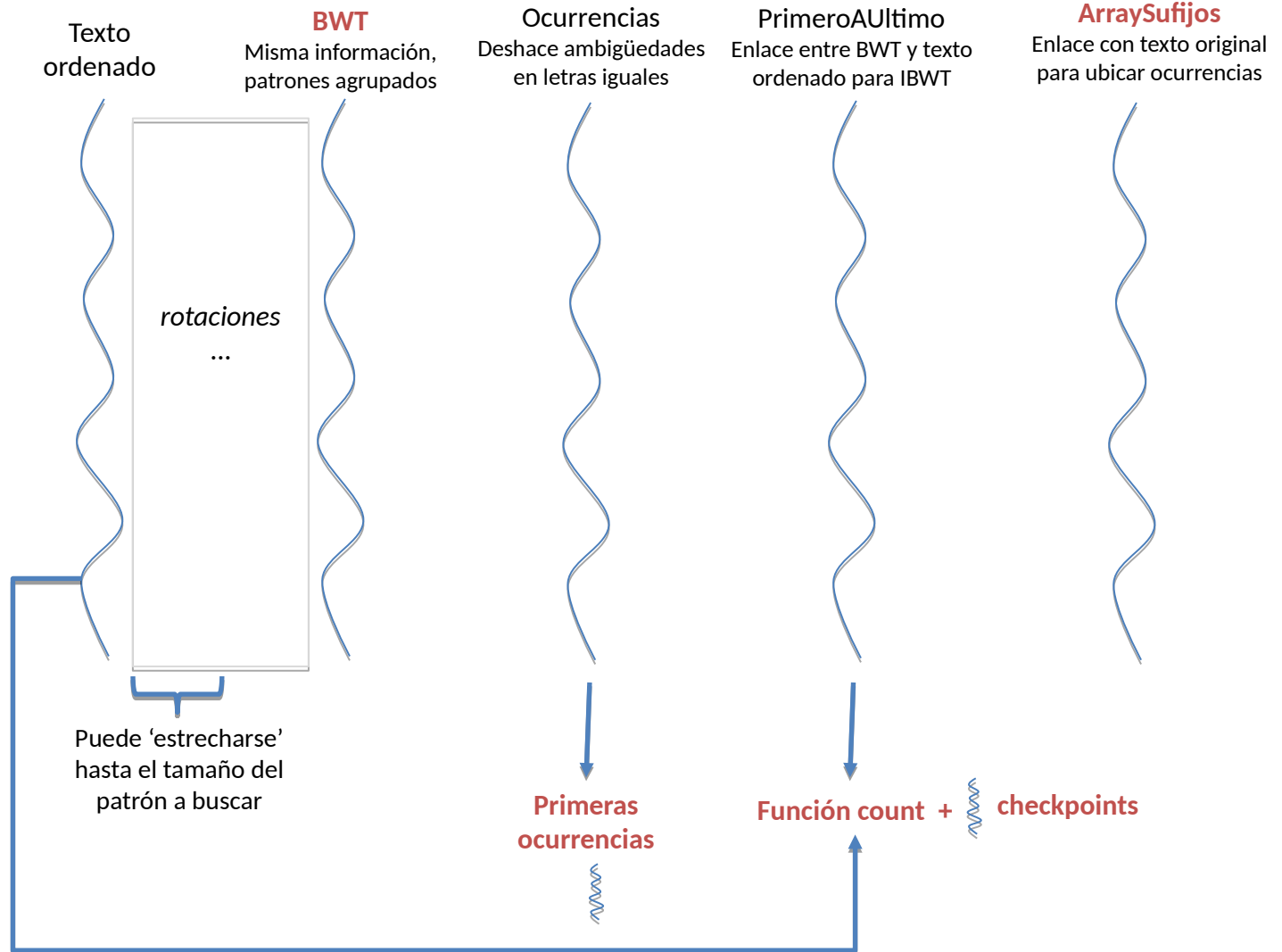
Ejercicio*

- Prueba:
 - text: genoma de *V cholerae*
 - pattern: g a t t a c a
 - d: 1
 - El resto de parámetros se pueden obtener a partir de text
 - Aunque el método debe calcular las posiciones de cada ocurrencia, sólo se pide como solución el número total de ocurrencias

*Enviar la solución encontrada (código y un pequeño informe sobre el rendimiento, complejidad, etc.) al correo del profesor

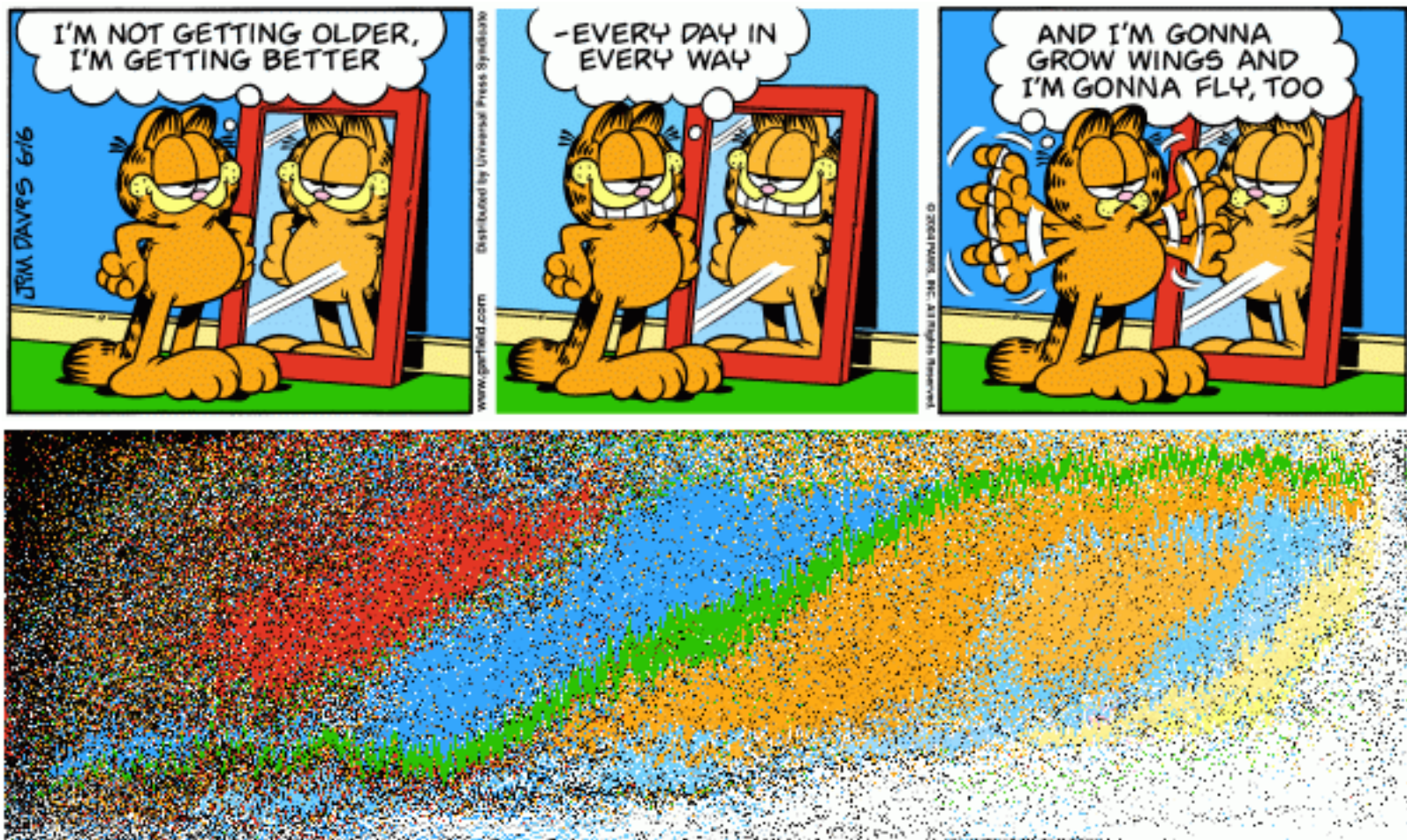
Resumen

Cada línea representa una estructura de datos de tamaño n (longitud del texto). Las marcadas en rojo son las que necesitamos mantener en memoria para hacer la búsqueda. Las líneas pequeñas implican mejoras que reducen el tamaño de la estructura a una longitud $\ll n$



Bibliografía

- Arrays de sufijos:
 - Manber, U., and Myers, G. (**1993**). Suffix arrays: a new method for on-line string searches. *SIAM Journal on Computing*, 22(5), 935-948.
- Transformada de Burrows-Wheeler
 - M. Burrows and D.J. Wheeler. *A block-shorting lossless data compression algorithm*. Systems Research Center, **1994**.



BWT aplicada a una tira cómica de Garfield
<http://www.mezzacotta.net/garfield/?comic=691>